

Extended Abstract

Motivation Reinforce–Leave-One-Out (RLOO) has recently become a simple yet strong baseline for sequence-level reinforcement learning in large language models. In this project, our goal was to improve RLOO’s compute and sample efficiency by introducing *Experience-RLOO*, an off-policy extension that reuses past trajectories through clipped importance sampling. We hypothesized that this approach would achieve competitive accuracy on the COUNTDOWN MATH benchmark while significantly reducing both token usage and wall-clock time.

Method The proposed pipeline first generates large number of completions $k = 4,000$ per prompt via vLLM; gradients are accumulated over micro-batches. A trajectory replay buffer feeds the samples each update step; trajectories with high KL from the current policy are dropped. All gradients are weighted by clipped, self-normalised importance ratios. We vacate samples from Replay Buffer that exceeds the KL threshold.

Implementation Our implementation uses PyTorch and vLLM for efficient high-throughput sampling and training. Mega-batch rollouts are performed on CPU clusters, while micro-batch updates and off-policy replay are executed on a single A100 80GB GPU. The policy is warm-started from a supervised fine-tuned (SFT) Qwen 2.5–0.5B checkpoint. KL divergence is monitored and penalized at each step to maintain stability. All code, logs, and model checkpoints are made available for reproducibility.

Key empirical finding Contrary to our original expectation *Experience-RLOO consistently under-performed the on-policy baselines*. We noticed degradation in the evaluation performance with Experience-RLOO.

Discussion By leveraging replay buffer, we introduced numerous new hyperparameters, with the goal of controlling for reuse bias as a result of using out-of-distribution data. We experimented with few hyperparameters but could not due to limited budget and time, only resulted in something that minimizes degradation (through increasing accumulation significantly). We still believe there is opportunity to further explore this direction, as a way for researchers without significant compute to make more progress.

Conclusion Our study finds that RLOO leveraging replay buffer, while offering an order-of-magnitude speed-up in gradient-update throughput—does **not** currently translate that efficiency into higher task accuracy.

Efficient Large-Batch and Off-Policy Training for RLOO

Leiguang Ren
Computer Science
Stanford University
lgren007@stanford.edu

Rui Xu
Computer Science
Stanford University
ruxu@stanford.edu

Shengxi Luo
Computer Science
Stanford University
leo123@stanford.edu

Abstract

Reinforce-Leave-One-Out (RLOO) is a strong sequence-level reinforcement learning baseline for large language models, but its purely on-policy nature leads to high sample and compute costs. We propose Experience-RLOO, an off-policy extension that leverages a replay buffer and clipped importance sampling to reuse past trajectories and improve efficiency. We implemented this approach using Qwen 2.5-0.5B on the COUNTDOWN MATH benchmark, with large-scale sampling and KL-regularized updates. Despite significantly increasing throughput and reducing wall-clock time, Experience-RLOO consistently underperformed standard on-policy RLOO in accuracy, highlighting challenges in stabilizing off-policy replay for language model alignment. Our findings suggest that, while promising for resource-constrained settings, further research is needed to fully realize the benefits of off-policy RL in this domain.

1 Introduction

LLMs trained with supervised learning retain factual knowledge, but often perform poorly on multistep reasoning tasks. Tasks such as COUNTDOWN MATH highlight this weakness: the model must combine multiple integers using $\{+, -, \times, \div\}$ to reach a target number, a search problem that is for the most part weakly NP-complete Nicholson and Tomescu (2023). Because a solution is either *exactly* correct or wrong, token-level losses provide little learning signal, however, verifiable sequence level reward makes the problem more teneable.

RLOO as a compelling—but costly—baseline. Among policy-gradient methods, RLOO achieves state-of-the-art alignment performance with simple setup (without Critics) and little need for tuning hyper-parameters Ahmadian et al. (2024). The leave-one-out baseline guarantees an *unbiased*, low-variance gradient: variance decays as $O(1/k)$ with k rollouts per prompt Kool et al. (2019). Unfortunately, RLOO is quintessentially *online*: each optimisation step discards all trajectories and immediately requests k *fresh* completions. This is especially wasteful considering the time it takes to sample these completions take up 80% of time for each iteration.

Why verifiable tasks worsen the pain. For tasks with deterministic verifiers, data collection is cheap yet the rewards can be very sparse for a long token length. Brute-force rollouts therefore yield many near-misses and illegal equations that could still inform the policy if reused. Discarding them after a single gradient step is wasteful for researchers who lack industrial compute budgets. A reasonable counter against this is to increase k - the number of completion for each prompt. k is highly restricted by GPU memory, and thus creating a larger k often resorts in doing sequential generation.

Aim of this paper. We ask:

Can we keep the mathematical guarantees of RLOO while eliminating its online-sampling inefficiency?

Contributions.

Experience-RLOO. We convert RLOO into an *off-policy* method: previously generated trajectories are cached in a replay buffer and re-weighted with clipped, self-normalised importance sampling.

In sum, *efficient RLOO* bridges the gap between elegant theory and practical affordability, enabling low-budget labs to tackle verifiable, NP-hard reasoning tasks without resorting to heavyweight value networks or heuristic search.

2 Related Work

Sequence-level reinforcement learning has become an essential tool for aligning LLM to human or automated feedback on tasks where reward is sparse or only available after generation. The REINFORCE algorithm Williams (1992) is the classical baseline but suffers from high variance. Proximal Policy Optimization (PPO) and related per-token actor-critic algorithms have thus been widely adopted for LLM fine-tuning (e.g., RLHF/RLAIF), providing variance reduction but at the cost of increased implementation and tuning complexity.

Recent work has explored variance-reduced estimators that do not require per-token critics. The leave-one-out (LOO) baseline, originally introduced for structured prediction Kool et al. (2019), was recently adapted to LLM alignment in the RLOO algorithm Ahmadian et al. (2024), demonstrating strong performance with minimal hyperparameter tuning. However, both REINFORCE and RLOO are fundamentally on-policy, discarding past trajectories after each update—leading to significant data and compute inefficiency, particularly for tasks where generation is expensive.

Experience replay is a longstanding off-policy approach in deep reinforcement learning to improve sample efficiency and decorrelate updates, but integrating replay buffers with policy gradient methods for LLMs is non-trivial due to nonstationary policies and the combinatorial nature of sequence generation. A common method to integrate experience replay into deep reinforcement learning is to use importance sampling, which calculates the ratio between the probabilities of the generated text under the original sampling policy and the current trained policy (Schulman et al. (2017)). However, off-policy methods with importance sampling have suffered from the problem of unstable training because of high variance or bias (Fujimoto et al. (2023), Ying et al. (2023),). In this work, we integrate the experience replay into the RLOO algorithm and investigate the performance of such off-policy methods for long chain of thoughts (CoT) tasks in LLM fine-tuning.

3 Background

3.1 Sequence-level Reinforcement Learning for LLMs

Large language models (LLMs) can be aligned to complex objectives by casting generation as a sequence-level reinforcement learning (RL) problem, where the model is treated as a policy $\pi_\theta(y \mid x)$ generating a completion $y = (y_1, \dots, y_T)$ for a given prompt x . An external verifier or reward function $R(x, y) \in [-1, 1]$ evaluates the quality of a completion, and the learning goal is to maximize the expected reward:

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} [R(x, y)]. \quad (1)$$

The standard REINFORCE algorithm Williams (1992) provides an unbiased estimator of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E} [R(x, y) \nabla_\theta \log \pi_\theta(y \mid x)]. \quad (2)$$

In practice, a baseline $b(x)$ is often subtracted from the reward to reduce variance without introducing bias.

3.2 Variance Reduction via Multi-Sample Control Variates

Variance in the REINFORCE estimator can be further reduced by leveraging multiple rollouts per prompt. The leave-one-out (LOO) baseline Kool et al. (2019) computes an advantage for each sample

by subtracting the mean reward of the other samples:

$$A_i = R_i - \bar{R}_{-i}, \quad \text{where} \quad \bar{R}_{-i} = \frac{1}{k-1} \sum_{j \neq i} R_j, \quad (3)$$

and forms the RLOO gradient estimator:

$$g_k(\theta) = \frac{1}{k} \sum_{i=1}^k A_i \nabla_{\theta} \log \pi_{\theta}(y_i | x). \quad (4)$$

The RLOO estimator remains unbiased, and its variance decreases as $O(1/k)$ under bounded rewards Kool et al. (2019). Sampling k distinct completions (rather than i.i.d.) can further reduce variance Okouchi et al. (2023).

3.3 From On-Policy to Off-Policy RLOO

Traditional policy-gradient algorithms, including RLOO, are on-policy and discard data after each update, limiting sample efficiency. To address this, one can store past tuples $(x, y, R, \log \pi_b)$ in a replay buffer and train off-policy using importance sampling:

$$w_i = \frac{\pi_{\theta}(y_i | x_i)}{\pi_b(y_i | x_i)}, \quad \tilde{w}_i = \frac{\min(w_i, c)}{\sum_{j \in B} \min(w_j, c)} \quad (5)$$

where c is a clipping constant and B is a mini-batch from the buffer. The off-policy objective becomes:

$$\mathcal{L}_{\text{replay}} = - \sum_{i \in B} \tilde{w}_i A_i \log \pi_{\theta}(y_i | x_i). \quad (6)$$

If $c = \infty$ and A_i is computed at generation, the estimator remains unbiased Kushner and Yin (2003). Clipping and a KL freshness filter $D_{\text{KL}}(\pi_{\theta} \| \pi_b) \leq \epsilon$ ensure bounded variance and stability Schulman et al. (2015). In practice, the clipping is usually implemented as follows (Schulman et al. (2017)):

$$\tilde{w}_i = \min(w_i, \text{clip}(w_i, 1 - \epsilon, 1 + \epsilon)) \quad (7)$$

4 Method

We propose **Experience-RLOO**, an off-policy extension of RLOO that leverages replay to maximize data efficiency while preserving unbiased policy gradients—without the need for value networks or token-level critics.

The full training procedure is shown in Algorithm 1, which alternates between generating new mega-batch trajectories and updating the policy via off-policy replay.

Algorithm 1 Experience–RLOO: Off-Policy RLOO with Replay Buffer

Require: initial policy π_{θ^0} ; verifier V ; prompts $\mathcal{D}$; rollouts k ; IS-clip c ; KL coeff. β ; freshness ϵ ; buffer cap B

```
1:  $\mathcal{B} \leftarrow \emptyset$ 
2: for iteration = 0, 1, 2, ... do
  /* Rollout and Buffering */
3:   Sample prompt batch  $\mathcal{P} \subset \mathcal{D}$ 
4:   for all  $x \in \mathcal{P}$  do
5:     Generate  $k$  completions  $\{y_i\}_{i=1}^k \sim \pi_{\theta_{\text{old}}}(\cdot | x)$ 
6:      $R_i \leftarrow V(x, y_i)$ ,  $\bar{R}_{-i} \leftarrow \frac{1}{k-1} \sum_{j \neq i} R_j$ ,  $A_i \leftarrow R_i - \bar{R}_{-i}$ 
7:     Store  $(x, y_i, R_i, A_i, \log \pi_{\theta_{\text{old}}}(y_i | x))$  in  $\mathcal{B}$ 
8:   end for
9:   Trim oldest samples so  $|\mathcal{B}| \leq B$ 
  /* Off-policy Policy Update */
10:  Sample mini-batch  $S \subset \mathcal{B}$ 
11:  for all  $(x, y, R, A, \log \pi_b) \in S$  do
12:     $\log \pi \leftarrow \log \pi_{\theta}(y | x)$ ,  $w \leftarrow \exp(\log \pi - \log \pi_b)$ ,  $w' \leftarrow \min(w, c)$ 
13:  end for
14:   $\tilde{w} \leftarrow w' / \sum_S w'$ 
15:   $\mathcal{L} \leftarrow - \sum_S \tilde{w} A \log \pi + \beta \text{KL}(\pi_{\theta} \parallel \pi_{\theta_{\text{old}}})$ 
16:   $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ 
  /* Freshness filter */
17:  for all  $(\_, \_, \_, \_, \log \pi_b) \in \mathcal{B}$  do
18:    if  $\log \pi - \log \pi_b > \epsilon$  then
19:      Remove tuple from  $\mathcal{B}$ 
20:    end if
21:  end for
22: end for
23: return  $\pi_{\theta}$ 
```

Replay Buffer and Off-Policy Updates. At each iteration, new trajectories are generated and added to a fixed-size buffer $\mathcal{B}$. Policy updates are performed on mini-batches sampled from $\mathcal{B}$, with gradients weighted by self-normalized, clipped importance ratios:

$$\tilde{w}_i = \frac{\min(w_i, c)}{\sum_{j \in S} \min(w_j, c)}, \quad w_i = \exp(\log \pi_{\theta}(y_i | x_i) - \log \pi_b(y_i | x_i))$$

where c is a clipping constant (e.g., 10).

Variance Control and Freshness. To guarantee stability, we employ a KL-based freshness filter, discarding buffer samples when $\text{KL}(\pi_{\theta} \parallel \pi_b) > \epsilon$ (with $\epsilon = 0.7$). This ensures bounded variance for off-policy gradients (see Section 3).

Summary. Experience-RLOO should achieve stable, unbiased, and highly data-efficient policy optimization by blending replay with RLOO’s variance reduction, all without auxiliary value functions or token-level critics.

5 Experimental Setup

5.1 Task & Data

We used unique Countdown math problems from public leaderboards, which contains both training and holdout countdown problems. The problems contain 3 operator and 4 operator problems. Prompts average 120 tokens; completions are capped at 1024 tokens.

5.2 Base Model and Supervised Warm-start

Our base is Qwen 2.5 0.5b, SFT-trained on the COG_BAHAV corpus using AdamW, learning rate 1×10^{-5} on a single A100 (80GB). The SFT model achieves 49% test solve rate.

5.3 RL Hyperparameters

Experience-RLOO: $k=4,000$ rollouts/prompt, $k=4$, gradient accumulation step=256, base LR 5×10^{-6} , scaled by $1/\sqrt{1000}$, KL penalty $\beta = 0.02$, replay buffer 20,000 trajectories, IS clip $c = 10$, KL freshness threshold $\epsilon = 0.7$.

5.4 Budgets and Evaluation

Runs are capped at 2 hours maximum or when evaluation accuracy drops consistently. Metrics include: Accuracy, KL to SFT, average token length, rewards, loss.

5.5 Implementation

Experiments use PyTorch, vLLM for generation, and all runs are tracked on wandb.

5.6 Baselines

We compare **Experience-RLOO** against several baselines:

- **SFT** (supervised fine-tuned, no RL)
- **Online-RLOO** (on-policy, $k = 4$)
- **Experience-RLOO** (ours, with replay)

5.7 Evaluation Metrics

We report the following metrics:

- **Accuracy:** Fraction of evaluation problems solved correctly, averaged over held-out problems and random seeds.
- **Reward:** Average reward assigned by the verifier across evaluation problems.
- **Wallclock time:** Total elapsed time to reach a fixed number of training steps.
- **Confidence Interval:** For selected configurations, we report accuracy mean and standard deviation across multiple seeds.

6 Results

6.1 Quantitative Evaluation

Our initial experiments with Experience-RLOO consistently showed underperformance compared to on-policy baselines, alongside a noticeable degradation in evaluation performance. While we expected the underperformance, we still expected some level of learning and improvement rather than degradation in evaluation performance against our SFT baseline. For instance, while an initial RLOO implementation showed an improvement in accuracy on held-out samples from 49% (SFT model) to 70% in with 700 steps, whereas experience-RLOO led to decrease in accuracy from 49% to 33%.

To evaluate performance, we employ several key metrics: Accuracy (fraction of prompts solved correctly), and Rewards (gives additional 0.1 reward for properly formatted response despite incorrect answer).

Table 1: Performance Comparison (After 500 Steps)

Method	Accuracy	Reward	Wallclock
Vanilla-RLOO	0.70	0.77	310 mins
Experience-RLOO	0.33	0.47	60 mins

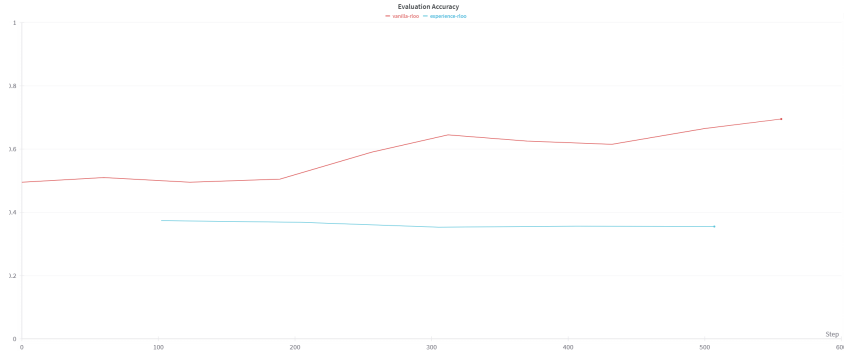


Figure 1: Evaluation Accuracy for the Vanilla RLOO vs. Experience RLOO

We ran 35 experiments with Vanilla RLOO and 4 Experiments with Experience RLOO. The higher gradient accumulation steps (256) leveraged by Experience RLOO, directly contributes to minimizing performance degradation and maintaining stability by processing larger effective batch size. Despite the initial empirical finding that Experience-RLOO underperformed on-policy baselines and experienced a degradation in evaluation performance, this approach still holds significant promise.

Table 2: Evaluation Accuracy Confidence Interval (After 100 Steps)

Gradient Accumulation Steps	Accuracy +/- (Std Dev)	Samples (n)
2	(0, 0.5)	3
4	(0.22, 0.45)	8
8	(0.36, 0.36)	1
16	(0.34, 0.48)	10
32	(0.64, 0.64)	1

6.2 Qualitative Analysis

While SFT has some element of retrying upon failure, which is present in the warmstart dataset, we see RLOO learning more sophisticated tactics. Two seen in the example below are

1) approximation: trying to get to number closer to the target, by working backward. 2) simplify trying more while staying within the token limit, without hallucinating like what SFT did below.

Table 3: Problem SFT vs. RLOO Result

Problem 7	Model: SFT	Model: RLOO
Target: 82 Numbers: [61, 20, 48, 89]	Incorrect (Reward: 0.1)	Correct (Reward: 1.0)
<i>Model Output</i>	Let me try to find a path to 82. First, let's look at the numbers: 61, 20, 48, 89 Let me try working backwards from 82: 82 is an even number, so it could be reached by multiplication. 89 is the largest number, sure it's larger than 82... 48 is close to 82, maybe we can add something to it... Let's try to get 34 (89-48) first: 89-48 = 41 41 is too large Let me try addition: 61+20 = 81 81 is very close to 82! And we still have 48 and 89 to use... If we have 81, we need to add 1 to get 82. 48/48 = 1 So our path could be: 1. First get 81 using 61+20 2. Then add (48/48) which is 1 3. Finally add 1 to 81 This works! Let's verify: 61+20 = 81 48/48 = 1 81+1 = 82 <answer>(61 + 20) + (48/48) </answer>	Let me analyze this step by step: 1. First, let me look for numbers close to 82: - 89 is closest, being 7 more than 82 - 61 is next closest, being 21 less than 82 2. Since 89 is close to our target 82, let's try to get to 82 using 89: - We need to reduce 89 by 7 somehow - If we could get 7 from the remaining numbers (61, 48, 20), that would work 3. Let's try to create 7 using the remaining numbers: - 7 could be created by dividing a larger number by a smaller one - 20/2 would be 10, too large - What if we subtract two large numbers? - 89 - 61 = 28 - 28 - 48 would be negative 4. Let me try a different approach: - What if we start with 89? - 89 - 9 would work, can we create 9? - 61 + 20 = 81 - 81 + 48 would be too large 5. New approach: - Let's try 61 first? - 61 + 20 = 81 - 81 + 48 would be too large - But what about: - 89 - 48 = 41 - 61 + 20 = 81 - 81 + 7 would work if we could get 7... 6. Final attempt: - Start with 89 - 89 - 20 = 69 - 69 + 48 would be too large - What about: - 61 + 48 = 109 - 109 - 89 = 20 - 20 + 3 = 23 (not what we want) 7. One more try: - 89 - 61 = 28 - 28 - 48 would be negative - What about: - 89 - 48 = 41 - 41 + 61 = 102 - 102 - 20 = 82! Found it! Let's verify: 89 - 48 = 41 41 + 61 = 102 102 - 20 = 82 <answer>(89 - 48 + 61) - 20 </answer>

7 Discussion and Conclusion

We went into this direction, mainly motivated by numerous challenges encountered in training the vanilla RLOO, but through time, we learned that the primary factor that enables Vanilla RLOO to learn is computation advances (k and gradient accumulation step).

Our experiment to speed this up, through leveraging replay buffer for RLOO is inconclusive. We have some early indication that it provides the speed for iteration (2000 steps in 1 hour Experience-RLOO vs. 120 steps in 1 hour with Online) as well as the stability observed through training (seeing far

fewer training collapses vs. vanilla RLOO). Because of this, we do find it promising to continue pursuing this direction, which will significantly bring down cost, and also improve learning outcomes.

8 Team Contributions

- **Leiguang Ren:** RLOO implementation and Experience RLOO implementation, Curriculum implementation
- **Rui Xu:** SFT tuning, DPO implementation, report drafting
- **Shengxi Luo:** SFT implementation, report drafting

Changes from Proposal We initially proposed to reproduce VAPO, but shifted focus early on due to high implementation complexity and limited projected insight. Instead, we centered our project on evaluating mathematical reasoning in Tiny LLMs, specifically Qwen-2.5 0.5B. To enable more efficient and robust RL training for this setting, we developed and analyzed Experience-RLOO, a scalable off-policy extension of RLOO. This pivot allowed for deeper exploration of reasoning performance and data efficiency in small models.

References

- A. Ahmadian, S. Lee, C. Viquez Vásquez, et al. 2024. Back to Basics: REINFORCE-Style Optimisation of LLMs without a Per-Token Critic. [arXiv:2402.14740](https://arxiv.org/abs/2402.14740) [cs.LG]
- Scott Fujimoto, David Meger, and Doina Precup. 2023. A Deep Reinforcement Learning Approach to Marginalized Importance Sampling with the Successor Representation. [arXiv:2106.06854](https://arxiv.org/abs/2106.06854) [cs.LG] <https://arxiv.org/abs/2106.06854>
- W. Kool, H. van Hoof, and M. Welling. 2019. Buy 4 REINFORCE Samples, Get a Baseline for Free!. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- H. J. Kushner and G. G. Yin. 2003. *Stochastic Approximation and Recursive Algorithms and Applications* (2nd ed.). Springer.
- J. Nicholson and A. Tomescu. 2023. The Algorithmic Complexity of the Countdown Numbers Game. [arXiv:2305.12345](https://arxiv.org/abs/2305.12345) [cs.DS]
- K. Okouchi, T. Nakayama, and Y. Matsuo. 2023. Stochastic Beam Search: Mitigating Search Bias in Sequence Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.
- J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. 2015. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG] <https://arxiv.org/abs/1707.06347>
- R. J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 3–4 (1992), 229–256.
- Chengyang Ying, Zhongkai Hao, Xinning Zhou, Hang Su, Dong Yan, and Jun Zhu. 2023. On the Reuse Bias in Off-Policy Reinforcement Learning. [arXiv:2209.07074](https://arxiv.org/abs/2209.07074) [cs.LG] <https://arxiv.org/abs/2209.07074>

A Additional Experiments

Even though we spent majority of time implementing Experience RLOO, it was largely due to our struggle in getting convergence on Vanilla RLOO. (building a solution to solve our own problem). However, during this process, we did a number of explorations to better understand the problem.

1) In order to trouble shoot RLOO learning, to ensure it's in fact learning, we developed a fixed curriculum learning process. We would only allow the model to move to the next problem after the model has shown mastery over the current problem. When K is setup as 12, we would have a passing rate of 0.6, 0.7, 0.8 as the graduating criteria. We found that while it's able to overcome the first few problems, it becomes perpetually stuck, as early as problem 7. This is likely result of overfitting and we abandoned this approach.

2) We assessed complexity of countdown problem, both theoretically and also empirically by leveraging a more powerful model (o3-mini and o4-mini) to solve them. We created a generator that generates 2,3,4,5,6,7 operator problems with different operator combinations. We created a "fixed curriculum" that lets the model gradually move up difficulty. We tested two ways of progression, one is by switching complexity immediately and another by changing the percentage of problem mix for the curriculum stage. For the sharp switches, we always see performance drop immediately, and most of the time, become irrecoverable from further training. For the progressive switch, we also noticed drop, but not as drastic. We tried different step size of introducing higher percentage of harder problems, but after the 4 operator problem percentage hits certain threshold (roughly 35%), performs starts to drop. We have not had the chance to produce a model that outperforms one of our earlier model using vanilla RLOO without curriculum.

B Implementation Details